

Product Requirement Document

Phase 1

DOCUMENT TYPE: System Requirement

STATUS: Draft

AUTHOR: Ishan Awasthi

1. OBJECTIVE & SCOPE

- Automatic extraction of candidate data from resume documents and transformation into a structured format (JSON) to facilitate querying, analytics, and indexing.
- Produce a clear and robust report of the extracted candidate data.

Scope Definition: Phase 1 focuses on constructing a backend Python utility with basic Streamlit based UI. It establishes the parsing engine without the overhead of infrastructure and persistent storage layers. This approach is to ensure delivery of the core extraction logic while maintaining architectural simplicity.

2. SYSTEM ASSUMPTIONS

Because the final deployment environment and hosting infrastructure remain To Be Determined, the architecture is reliant on the assumptions listed as follows:

- **Lightweight Web Interface:** The core parsing logic will be built as a modular backend utility, but it will be wrapped in a lightweight Streamlit web app for Phase 1. This allows immediate uploading of the file and visualization of JSON output without the use of a terminal.
- **Loose Coupling:** The parser will maintain zero dependencies on specific persistent datastores. This ensures it can be seamlessly integrated into any internal database (SQL) or frontend application in future development phases.
- **Statelessness:** The system will not retain state between invocations. Each resume uploaded to streamlit app triggers an isolated parsing request. Returning a structured response synchronously without saving the data permanently.

3. INPUT CONSTRAINTS

To ensure system stability and predictable execution during Phase 1, following boundary conditions are applied on all incoming data payloads.

System Limitations & Bounds

- **Allowed File Formats:**

The system will strictly accept **.pdf (Portable Document Format) and .docx (Office Open XML)** extensions. Any other types or extensions will be immediately rejected with a **400 Bad Request error**.

- **Maximum File Size:**

Payloads are capped at a hard limit of **5MB** per document. This constraint is to prevent memory exhaustion, mitigate and cap LLM context window token limits.

4. PROCESSING PIPELINE

The core parsing engine utilizes a five-step execution pipeline:

Step 1: UI Data Input(Streamlit)

The workflow begins in the Lightweight Streamlit web interface. The user uploads a candidate's resume document (**.pdf/.docx only. Under 5MB**). The Streamlit front-end will securely buffer the binary file to temporary memory and pass it to the backend data extraction logic.

Step 2: The Text Extraction

The program ingests the binary file and extracts raw, unstructured UTF-8 text. This layer takes away the complexities of the underlying file formats by utilizing open-source libraries. **Pypdf** or **pdfplumber** will handle **.pdf** streams, while **python-docx** will parse **.docx** trees. Extracted text is stripped of non-printable characters and normalized before proceeding into the next stage.

Step 3: LLM Structuring

The normalized text is securely processed by a local, open-weight LLM:

- **Model Environment:** The pipeline utilizes a locally deployed **7.3 Billion parameter model (Mistral-7B-Instruct)** containerized via **Ollama**.
- **Hardware Footprint:** The model operates entirely within local VRAM (~4GB quantized footprint), requiring no external API calls, network dependencies, or extra cloud subscription fees.
- **Prompt Architecture:** This step utilizes strict Prompt Engineering. A carefully tuned System Prompt instructs the model to act strictly as a deterministic data extraction pipeline, outputting only JSON format data.
- **Determinism Controls:** Inference parameters will be restricted by locking the model temperature at **0.0**. This is to ensure non-creative outputs and to make the model map standard resume entities (e.g., Contact Info, Education, Experience, Skills) directly to our predefined schema keys without unnecessary conversational fluff.

Step 4: Schema Validation

The raw JSON response from the LLM is intercepted and validated against a predefined schema definition (using Pydantic or standard JSON Schema). This ensures quality and accuracy of data extraction and parsing. Missing mandatory fields will be replaced with null values, and hallucinated keys will be removed. Only complete and validated objects are returned to the caller.

Proposed JSON Scheme:

```
{  
  
  "candidate_profile": {  
  
    "personal_information": {  
  
      "first_name": "John",  
  
      "last_name": "Doe",  
  
    }  
  }  
}
```

```
    "email": "johndoe@email.com",

    "phone_number": "+91-1234567890",

    "location": "Bangalore, Karnataka, India",

    "linkedin_url": "https://linkedin.com/in/johndoe",

    "github_url": "https://github.com/johndoe",

    "portfolio_url": null

},

    "Professional_summary": "Highly motivated software engineering intern
with experience in Python and data pipelines...",

    "skills": {

        "programming_languages": ["Python", "JavaScript", "SQL"],

        "frameworks_and_tools": ["Git", "Docker", "React", "RestAPI"],

        "soft_skills": ["Technical Writing", "Agile Methodology"]

    },

    "work_experience": [

        {

            "company_name": "AI Tech Corp",

            "job_title": "Software Engineering Intern",

            "location": "Remote",

            "start_date": "2025-06",

            "end_date": "Present",

            "is_current": true,

            "responsibilities": [

                "Developed a resume parser utility using Python and LLMs.",

                "Reduced manual data entry time by 40%."

            ]

        }

    ]

}
```

```
    }

  ],

  "education": [

    {

      "institution_name": "University of Technology & Science",

      "degree": "Bachelor of Science",

      "major": "Computer Science",

      "start_date": "2024-09",

      "end_date": "2028-05",

      "gpa": "8.9"

    }

  ],

  "certifications": [

    {

      "name": "AWS Certified Cloud Practitioner",

      "issuing_organization": "Amazon Web Services",

      "issue_date": "2024-11"

    }

  ],

  "projects": [

    {

      "project_name": "Automated Deployment Script",

      "description": "A bash script to automate Docker container deployment.",

      "technologies_used": ["Bash", "Docker", "Linux"]

    }

  ]

}
```

```
]
}
}
```

Step 5: UI Report Rendering:

Once validated, the Python script will pass the JSON object back to the Streamlit UI. The UI will parse the dictionary and present a clean, human-readable format summary report(Will utilise Streamlit columns, metrics, dataframes). This is so the user can easily review the extracted data from the resume.

5. OUT OF SCOPE FOR PHASE 1

To prioritize the delivery of the core engine and establish a baseline, the following features are decided as out of scope for Phase 1:

- **Complex Production UI:** While a lightweight Streamlit app will be made for demonstration and testing, a full production-grade UI is out of scope for Phase 1.
- **Database Persistence Layer:** The system will not write to PostgreSQL, MongoDB, S3, or any other datastore, as data is processed in-memory.
- **Optical Character Recognition (OCR):** Scanned images and image-based PDFs without a text layer will not be processed.
- **Automated Workflows:** Email server integration and automated webhook triggers are not implemented in Phase 1